Pointer-First Agent Constitution: Architectural Analysis and Certification Report for Stela

1. The Crisis of Context and the Imperative for Referential Integrity

The evolution of repository-resident Artificial Intelligence has precipitated a fundamental crisis in governance: the tension between the fluidity of generative logic and the rigidity required for structural integrity. In the Stela ecosystem, this tension is not merely a theoretical concern but an operational bottleneck. The "Operator" (human authority) and the "Integrator" (AI lead) operate within a high-entropy environment where the slightest divergence in instruction sets—"Drift"—can cascade into systemic failure. The prevailing methodology of embedding verbose, repetitive instructions directly into task definitions has rendered the system effectively "un-serviceable." It creates a bloated context window where hallucinations flourish, and conflicting versions of the "truth" compete for the agent's processing power.

This report establishes the theoretical and practical framework for a "Pointer-First Agent Constitution." This paradigm shift moves the repository from a text-heavy, instruction-based model to a sparse, reference-based architecture. Under this constitution, no canonical text is duplicated. Instead, every operational entity—whether an Agent, a Task, or a Skill—functions as a pointer to a immutable "Single Source of Truth" (SSOT) defined in the Policy of Truth (PoT.md). This document provides an exhaustive analysis of the current system state [1], deconstructs the specific failures of legacy task structures, and proposes a rigorous certification regime to ensure that the B-TASK-XX system is compliant with the highest coding standards and the existential mandates of Stela governance.

The analysis proceeds from the axiom that "Context is not memory; it is a file".[1] If the context is cluttered with redundant behavioral definitions, the agent's cognitive capacity for payload execution is diminished. By enforcing a pointer-first discipline, we maximize the "Signal-to-Noise" ratio within the Dispatch Packet (DP), ensuring that the agent's "Contractor" persona executes logic with absolute literalism and linguistic precision, free from the interference of "Ghost Canon"—outdated rules lingering in the text of old tasks.

---

2. Constitutional Foundations: The Policy of Truth (PoT)

The stability of the Stela platform rests entirely upon the "Policy of Truth" (PoT.md), which functions not as a guideline but as the physical law of the repository. To understand why a pointer-first architecture is necessary, one must first analyze the constitutional axioms that render legacy structures obsolete.

2.1 The Doctrine of Preventative Governance

The PoT.md file explicitly states, "The system prevents action that violates policy. It does not request compliance".[1] This distinction is critical and serves as the primary differentiator between Stela and standard agentic frameworks. A system that *requests* compliance relies on the agent's interpretation of inline instructions, effectively asking the model to "please be good." A system that *prevents* violation requires a mechanism to check actions against an immutable standard before they are committed to the ledger.

In a legacy task structure, a prompt might include the instruction: "Do not use contractions." If the agent fails to adhere to this, the error is often caught only after the artifact is generated. In the pointer-first constitution, the task points to tools/lint/style.sh. The agent cannot complete the task without passing this binary gate. The governance is shifted from the *prompt* (probabilistic) to the *pipeline* (deterministic). This fulfills the "Preventative" doctrine. If a task file contains a copy of the "Behavioral Logic Standard," and the standard in PoT.md is subsequently updated (e.g., to tighten restrictions on "Contraction Prohibition"), the task file immediately becomes a "Ghost Canon"—a competing source of truth that contradicts the actual law. This creates a "Split-Brain" scenario where the Agent follows the task's outdated instructions while the Integrator enforces the new PoT.md. This is the definition of system failure.

2.2 The "Filing Doctrine" as a Semantic Map

The constitution establishes a rigid semantic separation of concerns through its Filing Doctrine, which must be respected by all B-TASK entities [1]:

- ops/ is defined as **Run**. It contains executable logic, binaries, and automation. It is the "hands" of the system.
- docs/ is defined as **Explain**. It contains manuals, rationale, and governance. It is the "memory" of the system.
- projects/ is defined as **Work**. It is the payload code where value is generated.
- storage/ is defined as **Trash**. It contains ephemeral artifacts, explicitly excluded from canon.

Legacy tasks that embedded execution logic (which belongs in ops/) or lengthy explanations (which belong in docs/) violated this doctrine. A pointer-first task respects this separation by containing *only* the coordination logic necessary to link a specific objective to the correct ops/ tools and docs/ references. It does not "do" the work; it points to the tool that does the work. This semantic map is enforced by the tools/verify.sh script, which validates the repository tree

against this doctrine.[1]

2.3 The Axiom of Context Hygiene

A distinct vulnerability identified in the repository architecture is the "Context Hazard." PoT.md explicitly defines a Context Hazard as "any inclusion of docs/library/agents, docs/library/tasks, or docs/library/skills in the global context manifest is a failure".[1] The global context (ops/lib/manifests/CONTEXT.md) is loaded at the start of every session via ops/bin/open. If the library of all possible tasks and skills were included here, two specific failures would occur:

1. **Token Saturation:** The context window would be flooded with irrelevant instructions (e.g., how to perform a database migration when the task is a CSS update), degrading the Agent's reasoning capacity for the specific task at hand.
2. **Recursive Drift:** If the context defines the agents, and the agents define the context, a self-referential loop is created that makes it impossible to isolate variables during debugging.

The pointer-first constitution resolves this by strictly enforcing "Just-In-Time" (JIT) loading. Agents, Tasks, and Skills are never loaded globally. They are referenced via pointer only when the specific Dispatch Packet (DP) requires them. This is why the B-TASK files must be rigorously stripped of inline text; they must remain lightweight enough to be injected into the context *on demand* without displacing the actual project code.

---

3. Staffing Protocol: The Tripartite Governance Model

The "Staffing Protocol" outlined in the request is not merely a hierarchy of users; it is a division of cognitive labor that the B-TASK system must facilitate. The constitution defines three distinct roles, and the serviceability of a task depends on its ability to address the correct role.[1]

3.1 Operator (Human): The Source of Finality

The Operator owns "final decisions, approvals, and secrets." Crucially, the Operator "performs all commits, pushes, and merges." This constraint creates a hard boundary for the AI agents. A task that includes instructions like "Commit the changes" or "Push to main" is structurally invalid—un-serviceable—because it commands the AI to perform an action reserved for the Operator.

In the pointer-first system, tasks must be written to produce *artifacts* (diffs, patches, shell scripts) that the Operator can review and execute. The B-TASK must point to the *creation* of a commit message (e.g., via ops/lib/scripts/git-prep.sh), not the *execution* of the commit. The "Mandatory Closing Block" in TASK.md [1] reflects this, requiring the agent to generate a

"Primary Commit Header" and "Squash Stub" for the Operator to use.

3.2 Integrator (Lead AI): The Guardian of Structure

The Integrator "maintains governance, structural integrity, and auditing." It generates Dispatch Packets and detects "Drift." This persona operates at the meta-level. The tasks assigned to an Integrator (e.g., B-TASK-04 Plan Command or B-TASK-08 Draft DP) must focus on the manipulation of the *governance surface* (TASK.md, SoP.md) rather than the *project surface*.

The Integrator relies on the ops/bin/dump tool to perceive the system state. If a task does not explicitly mandate a "Verification" step using ops/bin/dump, the Integrator is blind. Therefore, a serviceable task for an Integrator *must* include pointers to the dump binary and the tools/lint/truth.sh script to verify that its own planning has not violated the Policy of Truth.

3.3 Contractor (Guest AI): The Engine of Logic

The Contractor "executes specific logic tasks and drafts implementation details within a defined scope." This is the worker drone. The Contractor has the highest risk of introducing entropy (bad code, typos, contractions).

For a Contractor task (e.g., B-TASK-01 Code Review or B-TASK-05 Refactor), the "Behavioral Logic Standard" is the primary constraint. The task must enforce "Linguistic Precision" (no contractions) and "Absolute Literalism." A pointer-first task achieves this by referencing S-LEARN-04 (Coding Standards) and S-LEARN-05 (Security Protocols).[1] The Contractor does not need to "know" the security policy; it needs to know *where* the policy is, and be compelled to check its work against it.

---

4. Behavioral Logic Standard: Encoding Discipline

The prompt outlines specific parameters for AI behavior: Linguistic Precision and Operational Directives. These are not abstract ideals; they must be encoded into the B-TASK files via pointers to executable linters and specific S-LEARN modules.

4.1 Linguistic Precision as a Compile-Time Error

- **Contraction Prohibition:** The requirement to "Refrain from using contractions" ensures rhythmic precision. In the legacy system, this was a text instruction: "Don't use contractions." This invites failure because "Don't" is itself a contraction. In the pointer-first system, this is enforced by tools/lint/style.sh [1], which greps for \b(don't|can't|won't|it's)\b. The B-TASK simply points to the linter. If the Contractor uses a contraction, the linter fails, and the task cannot be marked complete.
- **Quantitative Reporting:** "Specify exact deviations... rather than subjective descriptions." This requires the Agent to use tools like git diff --stat or wc -l to generate

numbers. A serviceable task must point to S-LEARN-06 (Advanced Git Forensics), which contains the specific commands (git diff --numstat) to generate this data.
- **Absolute Literalism:** "Interpret instructions with total fidelity." This is enforced by the "Freshness Gate" in TASK.md.[1] The Agent must echo: "Loaded: PoT, SoP, CONTEXT, MAP." If the Agent paraphrases this (e.g., "I have loaded the files"), it fails the literalism check.

4.2 Operational Directives and Anti-Drift

- **Anti-Drift Governance:** "Logic or files not aligned with PoT.md coordinates represent a system failure." This requires the B-TASK to include a ## Scope Boundary section that explicitly explicitly forbids edits outside the Allowlist. The tools/verify.sh script [1] mechanically enforces the directory structure; a serviceable task points to this script as a mandatory pre-flight check.
- **Logic Conflict Resolution:** "If a task violates TRUTH.md invariants... cease operations." (Note: The prompt references TRUTH.md, but the repo dump uses PoT.md. This is a drift in the prompt itself. The pointer-first system resolves this by treating PoT.md as the SSOT. If the user says TRUTH.md, the Agent—following PoT.md precedence—resolves it to PoT.md or stops to ask for clarification, fulfilling the "Absolute Literalism" directive).
- **Equilibrium Maintenance:** "A task is considered complete only when... SoP.md is updated." This is the "Closeout" phase. Every B-TASK must point to S-LEARN-08 (Prune SoP and Regenerate Context Bundles). Without this pointer, the task leaves the system in disequilibrium.
- **Reuse-First Discipline:** "Cross-reference all proposals against existing templates in ops/." The task must point the Agent to ops/lib/project/SCAFFOLD.md or ops/lib/scripts/ before allowing the creation of new files.

---

5. The Registry System: Architectural Analysis

The repository dump reveals a consolidated registry system under docs/ops/registry/.[1] This registry is the backbone of the pointer-first constitution. A pointer is only valid if it resolves to an entry in this registry.

5.1 Agent Registry (docs/ops/registry/AGENTS.md)

The Agent library defines the available workforce. Each agent must be pointer-first to be compatible with the tasks.

Agent ID	Name	Role	Pointer-First Compliance Analysis

R-AGENT-01	architect	System design, technical decision-making.	**High.** Uses PoT.md and docs/GOVERNANCE.md as primary constraints. Serviceable for B-TASK-07 and B-TASK-04.
R-AGENT-02	code-reviewer	Quality, security, maintainability checks.	**High.** Points to tools/lint/* and S-LEARN-04 (Standards). Serviceable for B-TASK-01.
R-AGENT-03	doc-updater	Documentation and codemap maintenance.	**High.** Driven by docs/MAP.md and docs/MANUAL.md. Serviceable for B-TASK-06.
R-AGENT-04	integrator	Planning, DP generation, complex integration.	**Critical.** Generates DPs based on TASK.md. This agent is the "Drift Detector."
R-AGENT-05	refactor-cleaner	Dead code cleanup, consolidation.	**High.** Points to tools/verify.sh for safety. Serviceable for B-TASK-05.
R-AGENT-06	security-reviewer	Vulnerability detection (OWASP).	**Critical.** Enforces PoT.md security policies. Serviceable for B-TASK-01 (Support) and B-TASK-03

			(Support).

Insight: The "Integrator" agent (R-AGENT-04) is the most distinct. While the Operator (Human) is the ultimate authority, the Integrator agent is the "guardian of governance." It detects drift. Therefore, B-TASK files assigned to the Integrator must be exceptionally rigid in their pointers, referencing SoP.md and TASK.md essentially as read/write interfaces.

5.2 Skill Registry (docs/ops/registry/SKILLS.md)

The Skill system (S-LEARN-XX) is the mechanism for retaining institutional knowledge. PoT.md imposes a strict directive: "Contractor closeout skill harvesting uses ops/lib/scripts/skill.sh harvest for provenance".[1] This prevents "Knowledge Drift," where the format of lessons learned degrades over time.

Skill ID	Name	File Path	Usage Scope
S-LEARN-01	Verification Loop	docs/library/skills/S-LEARN-01.md	Production Payload. Essential for B-TASK-01 and B-TASK-02.
S-LEARN-02	Project Guidelines	docs/library/skills/S-LEARN-02.md	Production Payload. Defines the "Stela Stack" (Next.js/Python).
S-LEARN-03	Zenith Reference	docs/library/skills/S-LEARN-03.md	Production Payload. API/State patterns.
S-LEARN-04	Coding Standards	docs/library/skills/S-LEARN-04.md	Production Payload. Formatting/Linting.
S-LEARN-05	Security Protocols	docs/library/skills/S-LEARN-05.md	Production Payload. RLS/CORS/Auth.
S-LEARN-06	Git Forensics	docs/library/skills/S	Payload & Maint. Debugging/Hotspot

		-LEARN-06.md	analysis.
S-LEARN-07	Harvest/Promote Skills	docs/library/skills/S-LEARN-07.md	Payload Only. **Mandatory Closeout.**
S-LEARN-08	Prune SoP/Context	docs/library/skills/S-LEARN-08.md	Payload Only. **Mandatory Closeout.**

5.3 Task Registry (docs/ops/registry/TASKS.md)

The Task Registry serves as the menu of approved operations. The prompt implies these need certification. A "Serviceable" task must map one primary Agent to one or more Skills, utilizing the ops/ toolchain to effect change. If a task in this registry does not have a corresponding file in docs/library/tasks/, it is a "Ghost Task" and represents a governance failure.

---

6. Forensic Audit of Current Tasks (B-TASK-01 to B-TASK-10)

The prompt asserts: "in their current state, tasks are essentially un-serviceable." We must analyze the B-TASK files provided in the dump [1] to validate this claim or identify where the "Upgrade" mentioned in SoP.md (DP-OPS-0036) fell short.

B-TASK-01: Code Review

- **Structure:** Points to R-AGENT-02 (Code Reviewer).
- **Pointers:** References tools/lint/style.sh, tools/lint/truth.sh, tools/verify.sh. JIT Skills: S-LEARN-04, S-LEARN-05.
- **Serviceability Analysis:** This task appears highly serviceable *conceptually*. However, closer inspection reveals potential gaps:
  - **Stop Condition:** "Stop if it fails." This relies on the Agent interpreting "fail." A more robust pointer would be: "Exit code non-zero -> STOP."
  - **Drift Risk:** The "Execution Logic" steps are written in narrative text ("Run git diff..."). If the git flags change (e.g., enforcing --staged), the text becomes stale.
  - **Correction:** Replace narrative steps with a pointer to a script: "Execute: ops/lib/scripts/review.sh" (if it existed) or "Apply S-LEARN-01 Procedure."

B-TASK-03: Integrate Command

- **Structure:** Points to R-AGENT-04 (Integrator).
- **Logic:** Runs ops/bin/open and ops/bin/dump.
- **Serviceability Analysis:** This task is a wrapper for binaries. It is highly robust because the logic lives in ops/bin, not the markdown. If the dump format changes from ChatGPT to Claude, the binary handles it. The task simply says "run the binary." This is the **Gold Standard** for Pointer-First architecture.

B-TASK-07: Refresh + Discuss

- **Structure:** Points to R-AGENT-01 (Architect).
- **Logic:** "Require OPEN and dump artifacts... Discuss the requested topic."
- **Analysis:** This task is purely analytical. It requires docs/MAP.md as a reference. Its scope boundary ("Discussion only... no edits") is the vital constraint.
- **Un-serviceability Risk:** "Discuss" is a vague command. It violates "Quantitative Reporting."
- **Improvement:** "Generate an Analysis Report citing specific file paths and line numbers. Calculate the impact of proposed changes on the codebase topography."

B-TASK-08: Refresh + Draft DP

- **Structure:** Points to R-AGENT-04.
- **Logic:** "Use TASK.md as the template and draft the DP."
- **Analysis:** This is critical. TASK.md is a living surface. If the task text duplicated the DP template, it would drift immediately. By pointing to TASK.md, it ensures the draft always matches the current standard.
- **Serviceability Gap:** The task assumes the Agent knows *how* to fill the template. It should point to docs/MANUAL.md Section 2 ("Dispatch Packet Mechanics") for the specific rules on creating a valid DP (e.g., "No placeholder text").

The Verdict on "Un-serviceable"

The tasks are "un-serviceable" not because they are broken, but because they are **insufficiently rigorous** for an autonomous system. They still rely on the Agent "knowing" how to perform a "Review" or "Discussion." They lack the **quantitative feedback loops** required by the Behavioral Logic Standard. A task that says "Review the diff" is open to interpretation. A task that says "Run tools/lint/style.sh and report the number of contraction violations" is serviceable.

---

7. The Pointer-First Transformation: Proposed Modifications

To certify the system, we must refactor the B-TASK library to strictly adhere to the

"Pointer-First" schema. This involves stripping narrative text and replacing it with executable references.

7.1 The Standardized Task Schema

A serviceable B-TASK file must adhere to the following schema:

1. **Header:** Task ID and Name (Must match Registry).
2. **Provenance:** Auto-generated block capturing *when* and *why* this task version was created (DP-ID, SHA).
3. **Orchestration:**
    - **Primary Agent:** Pointer to R-AGENT-XX.
    - **Supporting Agents:** Pointers to R-AGENT-XX.
4. **Pointers (The Core):**
    - **Constitution:** Always PoT.md.
    - **Governance:** docs/GOVERNANCE.md.
    - **Contract:** TASK.md.
    - **Toolchain:** Explicit paths to ops/bin/ or tools/.
    - **JIT Skills:** Explicit paths to docs/library/skills/S-LEARN-XX.md.
5. **Execution Logic:**
    - High-level steps that reference the *skills* rather than redefining the steps.
    - Example: "Execute Review using S-LEARN-01" instead of "Run npm lint, then run tests..."
6. **Scope Boundary:** Explicit limitations on what files can be touched.

7.2 Specific Refactoring Proposals

Refactoring B-TASK-01 (Code Review):

- **Current:** "Run git diff HEAD and review the diff using..."
- **Proposed:**

  ### Execution Logic

  1. **Context Load:** Confirm loading of S-LEARN-04 and S-LEARN-05.
  2. **Forensics:** Execute S-LEARN-06 Procedure 1 (Hot Zone Detection).
  3. **Automated Gate:** Run tools/lint/style.sh. STOP on failure.
  4. **Manual Review:** Apply S-LEARN-04 (Standards) to diff.
  5. **Report:** Output quantitative findings (Count of errors, severity).

Refactoring B-TASK-05 (Refactor Clean):

- **Current:** "Run git status --porcelain and stop if..."
- **Proposed:**

  ### Execution Logic

1. **Pre-Flight:** Execute ops/bin/open check (freshness gate).
2. **Identification:** Execute S-LEARN-06 (Churn Detection) to identify targets.
3. **Modification:** Apply refactor.
4. **Verification:** Execute S-LEARN-01 (Verification Loop).
5. **Closeout:** Execute S-LEARN-08 (Prune/Regenerate).

7.3 Removal of Inline Code Blocks

Problem: Tasks containing blocks of code (e.g., a Python script snippet) are un-serviceable because the code is dead text.

Solution: Move the code to ops/lib/scripts/ or tools/. Replace the code block in the task with a pointer: "Execute: bash ops/lib/scripts/my-script.sh." This ensures the logic is version-controlled, lintable, and executable.

---

8. Operational Directives & Logic Standards: Enforcing Equilibrium

The "Behavioral Logic Standard" mandates "Equilibrium Maintenance." This is the concept that the system must return to a stable state after every operation. The pointer-first constitution enforces this through the "Closeout" phase.

8.1 The Closeout Loop

Every B-TASK must conclude with a mandatory pointer to the Closeout routine defined in TASK.md. This routine involves:

1. **Update SoP:** Recording the change in the history ledger.
2. **Prune:** Running ops/bin/prune to remove old artifacts.
3. **Refresh:** Running ops/bin/llms to update the context bundles.

If a task lacks this pointer, it is "Leaky." It leaves the system in a state where the llms.txt pointers might be stale (e.g., pointing to a file that was just deleted). This violates the "Anti-Drift Governance" directive.

8.2 Quantitative Reporting via Tooling

The prompt requires "Quantitative Reporting." The pointer-first system automates this. Instead of asking the Agent to "estimate" progress, the task points to tools/verify.sh. The output of this script (e.g., "Files checked: 96, Errors: 0") provides the exact quantitative data required. The Agent's role is simply to pipe this output into the RESULTS file.

---

9. The Ops Kernel: Analysis of Binary Enforcers

A constitution is only as good as its enforcement. The ops/ directory functions as the executive branch, enforcing the laws written in PoT.md through binary logic.

9.1 The "Library Guard" (tools/lint/library.sh)

This script [1] is the primary enforcer of the pointer-first system. It verifies referential integrity:

- **Forward Integrity:** Every R-AGENT-XX referenced in a task must exist in docs/library/agents/.
- **Reverse Integrity:** Every agent in the library must be registered in AGENTS.md.
- **Skill Integrity:** Every S-LEARN-XX pointer in a task must resolve to a valid file.

If B-TASK-01 points to S-LEARN-99 and S-LEARN-99 does not exist, the task is un-serviceable. The linter acts as a "compiler error" for governance, preventing the merge of broken chains of command.

9.2 The "Context Linter" (tools/lint/context.sh)

The PoT.md axiom "If it is not in the Context, it does not exist" drives this tool. It verifies that ops/lib/manifests/CONTEXT.md is accurate. Crucially, it **fails** if it detects a "Context Hazard" (inclusion of the full docs/library/ tree). This protects the system from "context flooding" where the Agent is overwhelmed by irrelevant task definitions.

9.3 The "Task Policeman" (.github/workflows/task_policing.yml)

This workflow detects changes to "Gate Surfaces" (e.g., ops/bin/open, docs/MANUAL.md). If a Gate Surface changes, TASK.md *must* be updated. This enforces the "Active Work Surface" axiom. You cannot change the tools without updating the contract that uses them.

---

10. Stress Testing and Certification: The Path to Equilibrium

To certify the tasks system as "Completely Serviceable" with Stela, we propose a rigorous "Stress Test Regime" executed via tools/test/agent.sh and manual verification loops.

10.1 Test Case: The "Drift" Injection

- **Method:** Deliberately introduce a conflict between PoT.md and a local instruction.
  - *Modify PoT.md* to strictly forbid "Markdown Headers > H3".
  - *Create a Task* that instructs "Create a document with H1 headers."
- **Expected Result:** The Agent, complying with the "Logic Conflict Resolution" directive in

PoT.md, must **STOP** and report the violation. It should cite PoT.md as the overriding authority.
- **Failure:** If the Agent executes the task and creates H1 headers, the "Pointer-First" hierarchy has failed (the Agent prioritized the Task over the Constitution).

10.2 Test Case: The "Dead Pointer"

- **Method:** In B-TASK-XX, reference a non-existent skill S-LEARN-999.
- **Expected Result:** tools/lint/library.sh must fail the build. The Dispatch Packet must be rejected.
- **Serviceability Metric:** No task with a dead pointer can ever reach the main branch.

10.3 Test Case: The "Context Explosion"

- **Method:** Attempt to add docs/library/tasks to ops/lib/manifests/CONTEXT.md.
- **Expected Result:** tools/lint/context.sh must detect the "Context Hazard" patterns defined in PoT.md and fail the build immediately.

10.4 Certification Checklist

For a Task to be certified "Serviceable with Stela," it must pass the following binary gates:

1. **Lint Pass:** tools/lint/task.sh returns exit code 0.
2. **Registry Sync:** Task ID exists in docs/ops/registry/TASKS.md.
3. **Provenance:** Contains valid Captured timestamp and DP-ID.
4. **Pointer Resolution:** All R-AGENT and S-LEARN references resolve to files.
5. **Scope Boundary:** Clearly defined ## Scope Boundary section preventing project-wide writes unless authorized.

---

11. Conclusion

The transition to a Pointer-First Agent Constitution transforms the Stela repository from a collection of text files into a **Deterministic State Machine**. By treating PoT.md as the immutable kernel and B-TASK-XX files as lightweight pointers, the system eliminates the entropy of information duplication.

The "un-serviceable" nature of legacy tasks—their tendency to drift, hallucinate context, and violate scope—is resolved not by writing *better* prompts, but by writing *fewer* prompts that point to *harder* truths. This architecture ensures that the "Integrator" and "Contractor" operate in perfect lockstep, bound by a shared, single source of truth that is mechanically enforced by the ops/ binary layer. The system does not hope for compliance; it renders violation architecturally impossible.

Appendix: Verified Architecture Data

Table 1: Verified Agent Registry (Pointer-First)

ID	Name	Provenance	Specialization
R-AGENT-01	architect	DP-OPS-0029	System design, scalability, technical decision-making.[1]
R-AGENT-02	code-reviewer	DP-OPS-0029	Quality, security, maintainability checks.[1]
R-AGENT-03	doc-updater	DP-OPS-0029	Documentation and codemap maintenance.[1]
R-AGENT-04	integrator	DP-OPS-0029	Planning, DP generation, complex integration.[1]
R-AGENT-05	refactor-cleaner	DP-OPS-0029	Dead code cleanup, consolidation.[1]
R-AGENT-06	security-reviewer	DP-OPS-0029	Vulnerability detection, remediation.[1]

Table 2: Verified Skill Registry (Pointer-First)

ID	Name	File Path	Usage Scope
S-LEARN-01	Verification Loop	docs/library/skills/S-LEARN-01.md	Production Payload
S-LEARN-02	Project Guidelines	docs/library/skills/S-LEARN-02.md	Production Payload

S-LEARN-03	Zenith Reference	docs/library/skills/S-LEARN-03.md	Production Payload
S-LEARN-04	Coding Standards	docs/library/skills/S-LEARN-04.md	Production Payload
S-LEARN-05	Security Protocols	docs/library/skills/S-LEARN-05.md	Production Payload
S-LEARN-06	Git Forensics	docs/library/skills/S-LEARN-06.md	Payload & Maint.
S-LEARN-07	Harvest/Promote Skills	docs/library/skills/S-LEARN-07.md	Payload Only
S-LEARN-08	Prune SoP/Context	docs/library/skills/S-LEARN-08.md	Payload Only

Table 3: Task-to-Agent Alignment (Orchestration Map)

Task ID	Name	Primary Agent	Key Pointers
B-TASK-01	Code Review	R-AGENT-02	tools/lint/style.sh, S-LEARN-04, S-LEARN-05 [1]
B-TASK-02	Verification Command	R-AGENT-02	tools/verify.sh, tools/lint/context.sh [1]
B-TASK-03	Integrate Command	R-AGENT-04	ops/bin/open, ops/bin/dump [1]
B-TASK-04	Plan Command	R-AGENT-04	ops/bin/map, ops/bin/llms [1]
B-TASK-05	Refactor Clean	R-AGENT-05	tools/verify.sh, tools/lint/style.sh [1]

B-TASK-06	Update Documentation	R-AGENT-03	ops/bin/map, docs/MANUAL.md [1]
B-TASK-07	Refresh + Discuss	R-AGENT-01	docs/MAP.md [1]
B-TASK-08	Refresh + Draft DP	R-AGENT-04	TASK.md (Template) [1]
B-TASK-09	Refresh + Conform DP	R-AGENT-04	TASK.md (Template) [1]
B-TASK-10	Refresh + Audit	R-AGENT-02	tools/lint/dp.sh, RESULTS [1]

Works cited

1. dump-full-main-eccc11128.txt